Integrated Intrusion Detection and Prevention System with Honeypot in Cloud Computing

Sajaan Ravji
Beaufort Securities
London, United Kingdom
sajaanlakhman@hotmail.co.uk

Maaruf Ali
Int Assoc for Educators & Researchers
London, United Kingdom
maaruf@ieee.org

***Abstract*—Most organisations are now migrating their services into the cloud to offer a more flexible, open, mobile and ubiquitous service. However, this also brings more exposure to security threats, cyber attacks and difficulties in reliability and safety. The proposed plan is to deploy a Honeypot in the Intrusion Detection and Prevention System (IDPS) model in order to guarantee enhanced performance, expanded level of security in the Cloud computing environment and reduction in the dangers to the Cloud environment - by focusing on the issue of how the information is stored in the Cloud. The proposed framework is viable and the model effective, in terms of utilisation of the coordinated IDPS and Honeypot by an organization. The framework described uses both Anomaly Detection (AD) and Signature Detection (SD) in collaboration, to identify different attacks and deny them access through the use of the proposed Intrusion Prevention System (IPS). The objective of this report is to highlight, recognize and entrap internal intruders by the use of the Honeypot.**

Keywords— intrusion detection system, IDS, intrusion prevention system, IPS, honeypot, anomaly detection, AD, signature detection, SD, firewall, insider threat, computer attack, network attack

I. Introduction

Honeypots play an important rôle in analysing the behaviour and nature of attacks being directed against computer network in the form of rogue packets and brute force techniques. The deployment of honeypots within a cloud network enhances the security monitoring mechanism. This research project deployed the honeypot in the intrusion detection and prevention system of a LAN (local area network). The aim was not only to prevent external hackers but also internal attacks. The honeypot was also used to keep a fully detailed log record of all Internet activity through it.

II. Cloud Computing and Security

A. Cloud Computing Security Characteristics

Reference [1] explained five characteristics of cloud computing that need to be considered in any holistic security system:

- Security – establising a private cloud helps an organisation to have a greater control over their own data;
- Consistency – use of redundant sites aid in data recovery and cloud migration;
- Maintainability – cloud computing by its nature is more open access, thus use of controlled access utilsing VPN for example is highly recommened;
- Executability – all transactions need to be rigorously and sringently checked to remove any ambiguity. It should also be appreciated that all this is usually undertaken through more vulnerable web services as the system interface;
- Readiness – always on and available means security monitoring processes cannot rest and must run continuoulsy.

Reference [2] also identified the need for protection from: viruses and worms; man-in-the-middle attacks; identity spoofing; eavesdropping/information disclosure; tampering and finally from differential analysis threats. Furthermore [3] pointed out the particular problems of the Cloud Computing environment of:

- Information unreliability – vast amounts of information is being generated in a relatively very short period of time which needs to be acted upon before becoming tampered and void;
- Protection Inconsistency – this is of particular concern in the decentralised environment of Cloud Computing. One way to alleviate this is to use secure time stamped certificates with prescribed lifetimes;
- Responsibility ambiguity – clear rôles need to be defined for the numerous clients and adhered to. There should be no ambiguity of what is expected from each service provider.

The IDS (Intrusion Detection System) of a Cloud security system must thus not only be able to deal with the above typcial Cloud Computing characteristics but also be able to recognise an obfuscated and encrypted attack. Not only that, but more sophisticated attacks must also now be detected which now utilise polymorphic coding to avoid leaving any telltale signatures and thus avoid detection.

B. Intrustion Detection System (IDS) and Intrusion Prevention System (IPS)

The need for a more sophisticated system beyond just the use of firewalls has been made even more necessary due to Cloud Computing [4]. An IDS detects and stops an attack over a system and needs to respond reactively because there is no time to install any patches nor updates. Reference [5] gives four categories of IDSs in the Cloud Computing Environment, these are: Distributed IDS, Hypervisor IDS, Host Based IDS and Network based IDS. The Host based Intrusion Detection and Network based IDS are available for

free, meaning it is open source applications such as "Snort". Reference [6] states the functions of an IDS to be the:

- Tracking of client strategy infringement;
- Evaluation of document trustworthiness and systems;
- Investigation of unusual activity in a system;
- Investigation of vulnerabilities or loopholes and configurations of the system;
- Observation and analysis of both the system and the user's system and
- Capacity to perceive the pattern of attacks.

[7] summarised the main intrusion to cloud systems as being: insider attacks which may consist of root attacks and also by exploitation of the backdoor channel attack. An IPS in the cloud computing environment sits inline on the system and monitors when an incident happens, it initiates an action, taking into account the recommended rules. This is dissimilar to an IDS, which does not sit inline and is not active rather it is passive. Since IPSs make detection, some consider them to be superior over traditional IDSs. Thus IPSs as a cloud security tool in the overall security setup could stop intrusion attacks. Reference [6] argues that the IPS has been created out of the IDS, in any case, the two are truly diverse security tools that have distinctive functionalities and qualities. Fig. 1, below, summarises the intrusion detection and prevention system activity lifecycle.

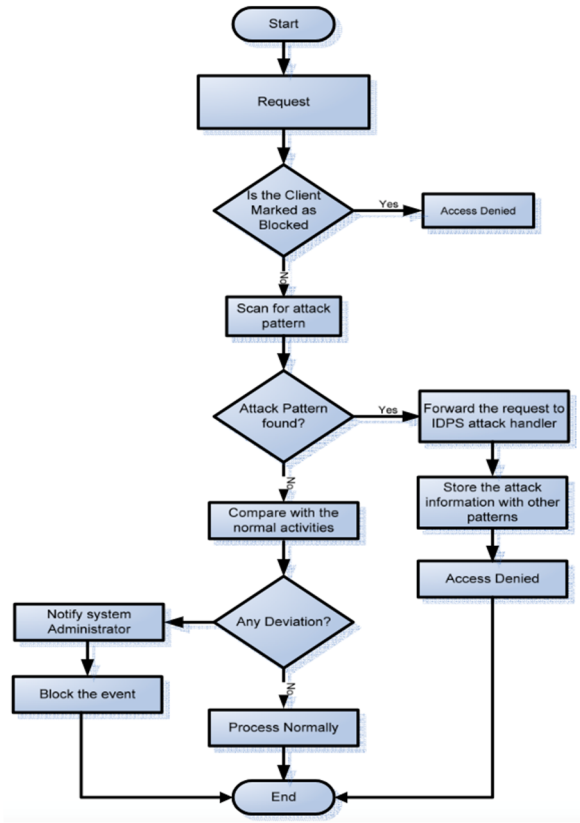

Fig. 1. Intrusion detection and prevention system activity. [6]

Fig. 1 shows the typical activities of an IDS and IPS system. Reference [6] explains that detection can by based on a behavioural strategy or be a knowledge based system (KBS) technique. If an abnormal behaviour that deviates from the norm is detected, then the network access is blocked. Computer reasoning is used to determine if the traces left by the potential attacker is indeed that of an attacker for a KBS.

C. *Cloud Intrusion Detection System (IDS) with Honeypot*

Honeypots are setup for detection, gathering information and prevention of attacks. They produce early warnings about threats and attacks. Honeypots are easy to utilise and record the required data. They are mostly utilised by corporate organisations to secure their systems from intruders. Honeypots are installed inside the firewall programmes. Accordingly, they can be better controlled. A honeypot is a security asset whose worth lies in being examined, attacked, or traded off. This implies that whatever is assigned as a honeypot, their *raison d'être* is to have the system tested, attacked and conceivably misused. The honeypot is a response and detection tool, instead of avoidance. Since honeypots cannot keep a specific intrusion or spread virus, it only gathers data and distinguishes the attack pattern. Subsequent to doing such, the defenders can react to this confirmation by building better resistances and countermeasures against future security threats [8]. The honeypot execute as a scrutiny and early cautioning tool. It is a network or a system site that gives off an impression of being the detached part of the system. It is purposely made to contain the data that is extremely significant to the programmers and hacker as a means of entrapment.

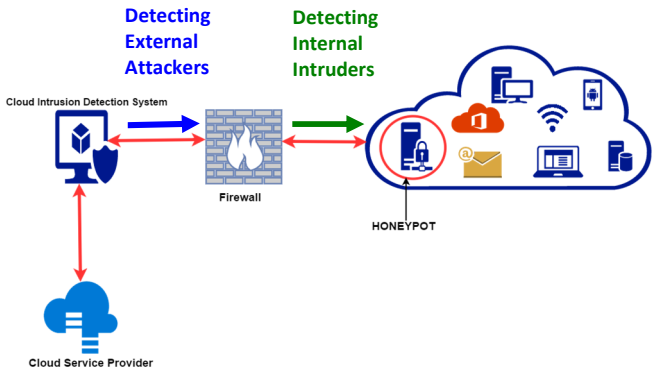

Fig. 2. Proposed design to detect both external and internal attackers.

The internal intruder rate is actually higher than the external attacks. Thus the only way to protect against internal intruders is to deploy a honeypot behind the firewall in the intruder detection and prevention system (IDPS) on a Cloud Computing Environment as shown in Fig. 2. Fig. 2, above, is also the proposed system of this research project. A simple IDPS may employ port scanning inside the Cloud Computing Environment. The proposal in [9] utilising a software defined network (SDN) and IPS managed to distinguish and block botnet/malware attacks. However, it failed to prevent a distributed denial of service (DDoS) attack that dwelled inside or outside the cloud. A solution to detecting DDoS was provided by [10] using a honeypot in the cloud computing environment. Furthermore Dr K. Curran [11] explained the advantages of utilising a cloud based honeypot in light of a cloud framework. As he said, "The advantages of using a cloud based honeypot on a cloud system is similar to traditional honeypots in that it should be able to vet whether a cloud system has been compromised or attempts were made to do so." [11]. It was shown in [12] that not any IDS or honeypot combination will work to

detect an attack. That in fact the right tools have to be matched so that the tools can collaborate together and give effective results. The research [12] also suggested that the tools should be chosen wisely with similar characteristics and behaviour in order to make the tools collaborate and produce effective results that would benefit the organisation.

III. Implementation Theory

OpenStack and Amazon EC2 were focussed upon in this research. In order to break down the information produced over the different instances in the honeypots, the following infrastructure was adopted.

The frontend infrastructure investigated the information gathered by the backend infrastructure. It was implemented utilizing PHP and the High charts graphing library for the representation of the attacker's information.

The backbend infrastructure gathered the information from different instances of the honeypots over the globe. The information generated and gathered was kept in a database for subsequent investigation. It was composed in the Python programming language. The primary modules included the:

DataLoader – this processed the raw information in the log, cleaned the database, handled and enhanced the pertinent data in the MySQL database to help facilitate investigation of the information further.

Extractor – this unit extricated the different log files such as the p0f (fingerprinting tool) log, the honeypot-specific and the honeypot databases from different instances of the honeypots. It was activated day-by-day. For safety it was duplicated from the remote instances of the honeypots to the machine locally.

Processor – this unit separated the significant data from the log file to investigate the honeypot information. The primary sub-modules for processing this information stage included, the:

- *Enricher* - The extricated information was further enhanced in this stage. For instance, for every IP address established in the p0f logs, the country of origin was found from the IP address. Likewise, the md5 hashes of the transferred malwares were examined utilizing the API to decide the type of the malware.
- *Parser* - The p0f log files were analysed to extract further information such as the systems' information and the attackers' IP addresses from the honeypot.

A. *p0f fingerprinting tool*

The p0f is an adaptable latent Operating System (OS) fingerprinting device. p0f can recognize the arrangement of machines that are associated with a container, the machines connected to it and even those machines that have to go through the firewalls. The use of p0f logs were used to extract the following accompanying information from the attacker:

- Time of occurrence of the attack;
- Port/Service of the attack on an attack-by-attack basis including also the Telnet and HTTP session information;
- The total time the attacker's machine was running;
- The IP address of the attacker.
- Attacker's method of connecting to the web, whether by use of Modem, Ethernet or DSL;
- The OS such as: Linux or Windows used by the attacker to perform the attacks.

B. *OpenStack*

OpenStack is a gathering of open source programming software that allows the network administrator to install and setup cloud processes and the storage framework. OpenStack began as a joint venture project between Rackspace Hosting and NASA. Rackspace contributed the cloud files coding to control the Object Storage part of OpenStack, while NASA contributed the Nebula coding to control the compute part. The OpenStack consortium now has more than 500 companies including, Citrix, Dell, Amazon EC2 (Elastic Compute Cloud), Microsoft Azure and so forth. There are five primary administration families under OpenStack:

- Horizon – UI (User Interface) dashboard services;
- Glance – Imaging service such as discovery, registration and delivery services;
- Nova – Compute service, cloud computing fabric controller;
- Keystone – Identity service, a central directory, a common authentication centre;
- Swift – scalable redundant storage service system.

C. *Nova – Compute service*

Nova is the computing fabric controller for the OpenStack cloud service provider. Everything needed to maintain the life cycle of every object instance created in the OpenStack cloud environment are taken care of by Nova. This also makes Nova a management platform that oversees: figure assets, system administration, approval and versatility needs of the OpenStack cloud. Nova does not give any virtualisation abilities without anyone else's input. Nova utilises the libvirt API to cooperate with upheld hypervisors. Nova uncovers its abilities through a web administration API that is mated with the EC2 API of the Amazon Web Services. Fig. 3, below shows the simplified OpenStack infrastructure.

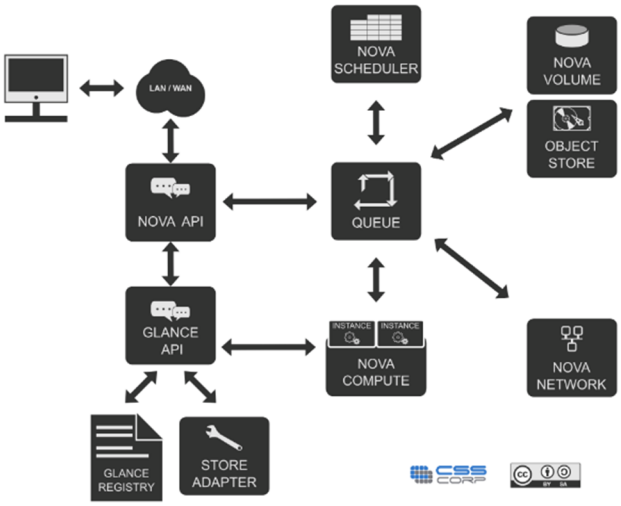

Fig. 3. OpenStack infrastructure [13].

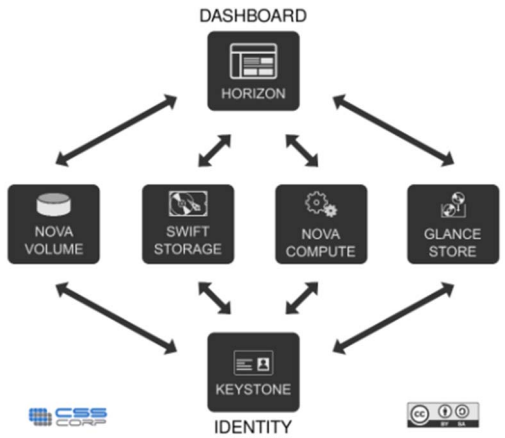

Fig. 4. Interaction between Keystone and Dashboard with the remaining OpenStack components [13].

D. *Features and functions*

Fig. 4 shows the key interactions between the Keystone identity module and Dashboard with the remaining key Open Stack components. The main features and functions of OpenStack can be summarised, as below:

- Managing the life cycle of the Instance created;
- Asynchronous operation in the long run to maintain a steady correspondence;
- Networking and Authorisation;
- Hypervisor rationalist support for: KVM, VMware, Xen, Hyper-V, vSphere, XenServer and UML;
- Managing of figure assets.

The proposed design was implemented using OpenStack as this was what the University of East London (UK) provided to its students at the time this research was conducted. The tool used for implementation from OpenStack also included: Ubuntu Server 14 and the CirrOS OS.

IV. Practical Implementation

The honeypot is deployed behind the firewall to present a new level of security. Amazon Web Service was not chosen as it is not open source and the security of the service is getting compromised regularly. The tools required to complete the implementation included: OpenStack as a Platform as a Service (PaaS), Ubuntu 14 Server, CirrOS OS, Windows 7 Professional and PuTTY (free SSH and Telnet client).

An instance of OpenStack was launched, using Ubuntu 14 as the instance where the honeypot was deployed. The instance was installed with the flavour 'm1.small', meaning it had a disk space of 20 GB and RAM memory of 2 GB. It was given an IP address 192.168.33.115 and an associated IP address of 172.16.0.114 for the SSH connection. The same procedural steps were also used to create another instance using the CirrOS OS with an IP address of 192.168.33.111 and an associated IP address of 172.16.0.121 for the SSH.

The instances of both can be accessed through SSH. For SSH, both Ubuntu and CirrOS must have a security key created, which is a public key and saved as the 'pem' file.

Before installing the honeypot on the Ubuntu server, it is highly recommended to update and upgrade the server with the latest updates of all the application and services. This was carried out using the command sudo apt-get update. Furthermore, an application known as 'git' had to be installed to enable the download of other software from the 'Github'. To install git type: apt-get install git –y

The open-source program 'Modern Honey Network' (MHN) from ThreatStream facilitates the methods of introducing and overseeing low-interaction honeypots. Furnished with MHN and access to an economical open cloud supplier, the experiments on honeypots could next be conducted. To install the MHN Server use the command: git clone https://github.com/threatstream/mhn.git

The next process was to run honeypot feeds on the MHN server by using the command: ./install_hpfeeds.sh This process could take up to 20 minutes to install. After hp-feeds is installed run the 'Mnemosyne' file by typing the command: ./install_mnemosyne.sh Again this could take up to 20 minutes. After the honeypot feeds and Mnemosyne is installed, the final step is to install the honeymap. To install the honeymap type in the command: ./install_honeymap.sh This process might take up to ten minutes to install.

'Ruby on Rails' standout amongst the most well-known application stacks for designers wishing to make websites and applications. The Ruby programming dialect, combined with the Rails improvement structure, makes application advancement straightforward in Linux, especially Ubuntu. Basically to run the PenTBox for Honeypot, install Ruby on Rails using the 'RVM' application by typing the command:

gpg --keyserver hkp://keys.gnupg.net --recv-keys 409B6B1796C275462A1703113804BB82D39DC0E3

\curl -sSL https://get.rvm.io | bash -s stable –rails

After Ruby and Rails is configured, the PenTBox suite from the sourceforge.net was downloaded using the command:

svn co https://pentbox.svn.sourceforge.net/svnroot/pentbox/trunk/ pentbox

PenTBox is a security suite that comprises security and dependability testing focused instruments for systems and networks. After downloading and installing the PenTBox suite, the directory was changed to PenTBox by typing cd pentbox then to run the Pentbox suite, type: ruby pentbox.rb

A lot of tools are provided in PenTBox such as: Cryptography tools, Network tools, Web, License and contact. To configure the honeypot, option two was selected for: 'Network tools'. Manual configuration was carried out next by selecting option two and setting the open port to 23. This was done because the configuration of the honeypot was deployed as a 'Port Scanner'. A Port Scanner is the least difficult type of honeypot. It listens for activity on ports generally filtered by 'black hats'. Its downside is that it actually cautions the internal attacker by allowing the link to the network and after that dropping it. This would make the intruder suspicious and prematurely end the attack. So a false message was inserted informing the hacker that the cloud was now secured. An alarm was set for the network administrator by typing 'Y' for 'if an attempt was detected'.

Next by observation of the screen output it was confirmed that the honeypot had indeed been configured correctly, was activated and was running smoothly. The

'honeypot.txt' log file would contain all the intrusion attempts.

To carry out an attack, an instance was created on OpenStack operating on the CirrOS OS. Basically by performing an SSH to 172.16.0.121, which is on IP address 192.168.33.111 and next executing a Telnet attack by typing the command telnet 192.168.33.115. This resulted in an 'intrusion attempt' being detected and a false message being sent to the attacker, that was set during the configuration of the honeypot. This clearly showed that the honeypot was successfully deployed. An attempt was also detected when an internal attacker of IP address 192.168.33.115 tried a Telnet attack on the target, which is the Ubuntu on IP address 192.168.33.115.

The intrusion attempt was also tried from an external network with a Class A IP address of 10.3.16.1. The honeypot deployed on the cloud computing environment (OpenStack) running Ubuntu server was successfully able to detect an intrusion to the server of IP address 192.168.33.115, which clearly explains that the proposed framework is able to detect external attackers also.

V. Evaluation

Fig. 5 clearly shows the performance improvement of attack detection on the number of packets for the proposed design of deploying the honeypot behind the firewall, in this case, in the Intrusion Prevention and Detection System (IDPS). Thus installation of the honeypot behind the firewall on the system is a good security measure. Even though the difference of having a honeypot and not having a honeypot in the cloud-computing environment is just 1% for an organization - that 1% is a big difference, as the preservation and integrity of the corporate data of the organisation is very important and cannot be compromised. At the point when the honeypots are set behind a firewall, it can introduce a new higher level of security to the internal network system, particularly if the internal network of an organization is not secured against the honeypot. It is vital to recognize the setup where the firewall allows access to the honeypot or when traffic from the Internet is not allowed through the network.

The honeypot is the designed to provide services while most of the services are not going through the Internet. They are not sent to the honeypot by the firewall. By setting the honeypot behind the firewall, it is certain to conform to the firewall rules if services from the Internet ought to be permitted. The proposed solution recognised any random attacks and contrasted it with the known attacks by signature detection and sent an alert to the IT administrator.

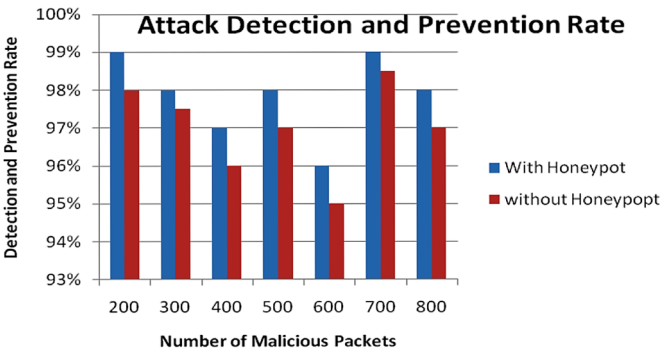

Fig. 5. Attack detection and prevention rate with and without a honeypot.

Firewall logs and system logs generate a vast amount of data - which can be quite overwhelming for human interpretation, as it is very hard to extract relevant information and value from the data if processed manually. On the other hand, honeypots gather little data but the data collected from the honeypot is very valuable. The honeypot idea of no normal data access generation when it is not an intruder accessing the system, significantly lessens the level of noise. Rather than logging gigabytes of information consistently, most honeypots gather a few megabytes of information for each day. A honeypot doubtlessly will detect an attack. Additionally, since such little data is gathered, then the identifying and collation of trends is what most organisations miss out on, unless automatically processed over a period of time by software.

Furthermore any additional security systems will face resource restrictions and exhaustion of resources will mean that the additional security process will no longer be functioning since its resources are no longer available. For instance, a firewall may freeze since its table of connection is full, it has come up short on assets, or it cannot screen and monitor the network traffic. This makes the firewall shut down all the connections rather than simply blocking unapproved traffic. While honeypots gather and monitor only malicious traffic, then characteristically it has no problems of exhaustion of resources unlike with the functioning of IDSs and firewalls.

An advantage of the honeypot operating in a resource-constrained environment is that the honeypot does not need the investment of a substantial amount of money nor actual hardware. Honeypots, as opposed to numerous security tools, for example, firewalls, do not require the latest level of technology, nor a large amount of RAM nor a large storage capacity in terms of say hard drives. Organizations can utilise the remaining PCs found in the company's IT junk room or the old portable workstation that the manager no longer uses. This implies a honeypot can be deployed on the latest gigabit networks using quite cheap used computing devices. Honeypots are also deployed and maintained with no extra cost as well.

The biggest advantage to the organizations of deploying a honeypot is that there is no development of any kind of algorithm and or a complex database to maintain. The main thing is that no configuration is required. Basically any employee with IT skills can deploy a honeypot for their organisation. The security experts do say that the simpler the configuration setup of the security measure the more reliable it is. The more complex the configurations, the more failures and breakdown of the hardware and software it is likely to occur as can be seen regularly with firewalls.

Expensive firewalls and Intrusion Detection techniques are bought from vendors, because of the extra strong security algorithm they purportedly offer. However, any lack of performance on real world networks can cause the vendor a serious problem, because the organisation that invested money found no attacks were detected so it will be seen as a disastrous investment of money and ruin for the reputation of the vendor too. Conversely, honeypots show their quality almost immediately upon setup. At whatever point they are attacked, organisations know the attackers are out there. By monitoring malicious traffic, honeypots can be utilised to legitimise their own particular quality as well as interests in other security assets also. At the point when the

administrator finds that there are no legitimate threats, honeypots can viably demonstrate that a lot of danger and actual threat exists were detected and intercepted.

VI. Conclusion

Honeypots give a new insight about the effectiveness of the use of such a simple security detection tool. The study of the log record can reveal a lot about the origins of the network attack and other valuable information. The honeypot can mirror the first attack and the captured event can be rerun many times over to further examine the procedural stages of the attack.

A low-interaction honeypot can easily be noticed by a professional hacker, whilst a high-level interactive one cannot be so easily noticed. Honeypots additionally gives an approach of cost effectiveness because rather than sniffing the entire network, honeypots detect just malicious activity in the cloud computing environment or the network. Intrusion protection solutions brought by honeypots are hard to disregard, particularly now as second generation honeypots are being used. In fact next-gen honeypots use tactical diversion-driven defences. These not only entice the attacker to its node, it actually keeps the attacker engaged whilst information is gathered in real-time about what tools and methods are being employed. The attacker can be allowed to continue for some time so that his/her goal can also be scrutinised and determined. In time, as organisations deploy multiple honeypots, these could turn into a fundamental technique for standard corporate level security operational procedure.

Honeypots can be utilised for detecting the attack, preventing the attack or reacting to an attack with the right measures. Honeypots can likewise be utilised for logging information of the threats, conducting research on the threats and the application of any defence measures to stop the attack. However, one vital point to be noted is that sole use of honeypots to secure a cloud computing is not suggested. The use of a honeypot in a demilitarised zone (DMZ) works just in the DMZ only, therefore, the entire cloud computing environment is not secured. This technique might not detect the internal attackers. In case the internal attacker attacks the server situated in the LAN, the honeypot set in the DMZ will definitely not be able to detect any malicious activity. Organisations need to have various security levels to effectively defeat the attackers. Hence consolidating honeypots with other security arrangements is paramount.

The main objective of this project has been achieved, which was detecting the internal attacker. It was successful because of the deployment of the honeypot in the cloud computing environment. In the proposed design, a low-interaction honeypot was deployed. However, for future work and further studies, a second generation high-interaction honeypot using tactical diversion-driven strategies can be deployed which will provide security to a variety of attacks in the wired, wireless or remote system. The outcome can be analysed further to demonstrate the deployment of high-interaction honeypots and though its use of bandwidth may increase - the success rate for the intrusion detected will be proportionally more.

References

[1] G. Aceto, A. Botta, W. de Donato and A. Pescapè, "Cloud Monitoring: definitions, issues and future directions", 2012 IEEE 1st Int. Conf. on Cloud Networking (CLOUDNET), Paris, France, 2012, pp. 63-67. DOI: 10.1109/CloudNet.2012.6483656. Available: https://www.researchgate.net/publication/260488124_Cloud_Monitoring_definitions_issues_and_future_directions [accessed 9th July, 2018].

[2] A. Malik and M. M. Nazir, "Security Framework for Cloud Computing Environment: A Review", J. of Emerging Trends in Computing and Information Sciences, Vol. 3, No. 3, March 2012, pp. 390 – 394. ISSN: 2079-8407. Available: http://www.cisjournal.org/journalofcomputing/archive/vol3no3/vol3no3_13.pdf [accessed 9th July, 2018].

[3] K. Lee, "Security Threats in Cloud Computing Environments", Int. J. of Security and its Applications, vol. 6, no. 4, Oct. 2012, pp. 25-32. Available: http://www.sersc.org/journals/IJSIA/vol6_no4_2012/3.pdf [accessed 9th July, 2018].

[4] S. Y. Ho, "Instrusion Detection – Systems for today and tomorrow". Available: https://www.sans.org/reading-room/whitepapers/detection/intrusion-detection-systems-today-tomorrow-341 [accessed 9th July, 2018].

[5] S. M. Moorthy and M. Rajeswari, "Virtual Host based Intrusion Detection System for Cloud", Int. J. of Eng. & Tech., Vol. 5, issue 6, Dec 2013/Jan 2014, p. 5024. Available: http://connection.ebscohost.com/c/articles/97719528/virtual-host-based-intrusion-detection-system-cloud [accessed 15th Jan, 2016].

[6] H. M. Alsafi, W. M. Abduallah and A. K. Pathan, "IDPs: An Integrated Intrusion Handling Model for Cloud Computing Environment", March 2012. Available: https://www.researchgate.net/profile/Hassen_Alsafi3/publication/221703248_IDPS_An_Integrated_Intrusion_Handling_Model_for_Cloud/links/0046353143a3bd5791000000.pdf [accessed; 9th July, 2018].

[7] C. Modi, D. Patel, B. Borisaniya, A. Patel and M. Rajarajan, "A survey on security issues and solutions at different layers of Cloud computing", The J. of Supercomputing, vol. 63, issue 2, pp. 561 – 592. DOI: 10.1007/s11227-012-0831-5 Available: http://openaccess.city.ac.uk/12199/7/RevisedPaperbyChirag.pdf [accessed 9th July, 2018].

[8] L. Spitzner, "The Value of Honeypots", 10th Jan., 2003. Available: http://www.informit.com/articles/article.aspx?p=30489 [accessed 9th July, 2018].

[9] N. F. Huang, C. Wang, I. J. Liao, C. W. Lin and C. N. Kao, "An OpenFlow-based collaborative intrusion prevention system for cloud networking", 2015 IEEE International Conference on Communication Software and Networks (ICCSN), Chengdu, 2015, pp. 85-92, 607 June 2015. DOI: 10.1109/ICCSN.2015.7296133

[10] K. Shridhar and N. Gautam, "A Prevention of DDoS Attacks in Cloud Using Honeypot", Int. J. of Science and Research (IJSR), vol. 3, issue 11, Nov. 2014, pp. 2378 – 2383. ISSN (online): 2319-7064. Available: https://www.ijsr.net/archive/v3i11/T0NUMTQxNTQ0.pdf [accessed 9th July, 2018].

[11] D. Winder, "How to use the cloud as a honeypot", 2nd Oct., 2014. Available: http://www.cloudpro.co.uk/cloud-essentials/cloud-security/4728/how-to-use-the-cloud-as-a-honeypot [accessed 9th July, 2018].

[12] V. Sing, A. Kumar and D. Kumar, "An Advanced Hybrid Intrusion Detection System in Cloud Computing Environment", Int. J. for Research in App. Sci. and Eng. Tech. (IJRASET), vol. 2, issue 6, June 2014, pp. 302 – 309. ISSN: 2321-9653. Available: http://www.ijraset.com/fileserve.php?FID=561 [accessed: 9th July, 2018].

[13] A Jha, Johson D, K. Murari, M. Raju, V. Cherian and Y. Girikumar, "OpenStack Beginner's Guide", CSS Corp, v. 3. Available: https://cssoss.files.wordpress.com/2012/05/openstackbookv3-0_csscorp2.pdf [accessed: 9th July, 2018]